Push Relabel algorithm

Push-Relabel approach is the **more efficient than Ford-Fulkerson algorithm. Goldberg's** "generic" maximum-flow algorithm runs in **$O(V^2E)$** time. **This time complexity is better than $O(E^2V)$ which is time complexity of Edmond-Karp algorithm** (a BFS based implementation of Ford-Fulkerson).
There exist a push-relabel approach based algorithm that works in $O(V^3)$

Similarities with Ford Fulkerson

- Like Ford-Fulkerson, **Push-Relabel also works on Residual Graph**

Differences with Ford Fulkerson

- Push-relabel algorithm **works in a more localized**. Rather than examining the entire residual network to find an augmenting path, **push-relabel algorithms work on one vertex at a time**
- In Ford-Fulkerson, net difference between total outflow and total inflow for every vertex (Except source and sink) is maintained 0. **Push-Relabel algorithm allows inflow to exceed the outflow before reaching the final flow. In final flow, the net difference is 0 for all except source and sink.**
- Time complexity wise more efficient.

The intuition behind the push-relabel algorithm (considering a fluid flow problem) is that we **consider edges as water pipes and nodes are joints**. **The source is considered to be at the highest level** and it sends water to all adjacent nodes. **Once a node has excess water, it pushes water to a smaller height node. If water gets locally trapped at a vertex, the vertex is Relabeled** which means its **height is increased.** Following are some useful facts to consider before we proceed to algorithm.

- **Each vertex** has associated to it **a height variable and a Excess Flow**. **Height** is used to determine whether a vertex can push flow to an adjacent or not (A vertex can push flow only to a smaller height vertex). **Excess flow** is the difference of total flow coming into the vertex minus the total flow going out of the vertex.

Excess Flow of u = Total Inflow to u - Total Outflow from u

- Like Ford Fulkerson. each edge has associated to it a **flow** (which indicates current flow) and a **capacity**

Following are abstract steps of complete algorithm.

Push-Relabel Algorithm

1) **Initialize PreFlow** : Initialize Flows and Heights

2) **While** it is possible to perform a **Push()** or **Relabel()** on a vertex
// Or while there is a vertex that has excess flow
Do Push() or Relabel()

// At this point **all vertices have Excess Flow as 0 (Except source**
// and sink)

3) **Return flow**.

There are three main operations in Push-Relabel Algorithm

Initialize PreFlow() It initializes heights and flows of all vertices.

Preflow()

1) Initialize **height and excess flow of every vertex as 0.**
2) Initialize **height of source vertex equal to total number of vertices in graph**.
3) Initialize **flow of every edge as 0**.
4) **For all vertices adjacent to source s, flow and excess flow is equal to capacity initially**.

1. **Push()** is used to make the flow from a node which has excess flow. If a vertex has excess flow and there is an adjacent with smaller height (in residual graph), we push the flow from the vertex to the adjacent with lower height. **The amount of pushed flow through the pipe (edge) is equal to the minimum of excess flow and capacity of edge.**
2. **Relabel()** operation is used **when a vertex has excess flow and none of its adjacent is at lower height**. We basically increase height of the vertex so that we can perform push(). To increase height, we pick the minimum height adjacent (in residual graph, i.e., an adjacent to whom we can add flow) and add 1 to it.

Illustration:

Residual graph is different from graphs shown. **Whenever we push or add a flow from a vertex u to v, we do following updates in residual graph :**

1. We **subtract the flow from capacity of edge from u to v. If capacity of an edge becomes 0, then the edge no longer exists in residual graph.**
2. We **add flow to the capacity of edge from v to u**.

For example, consider two vertices u an v.

In original graph

3/10

u ---------> v

3 is current flow from u to v and

10 is capacity of edge from u to v.

In residual Graph, there are two edges corresponding to one edge shown above.

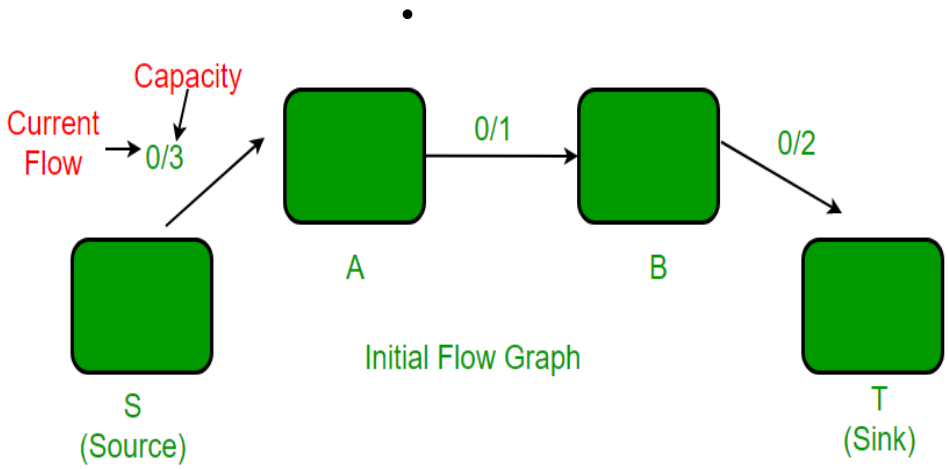
7

u ---------> v

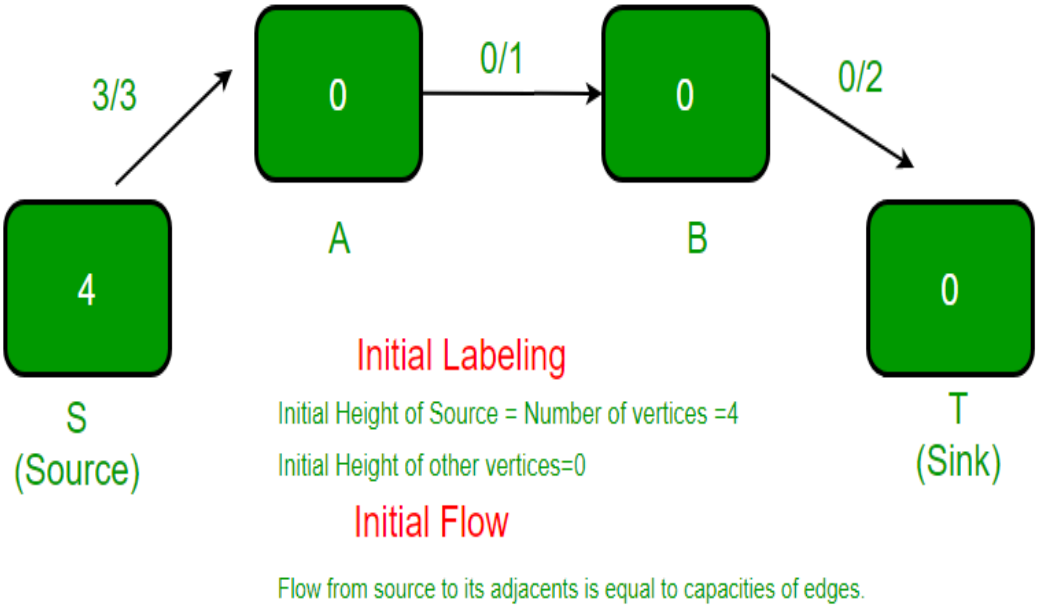
3

u <--------- v

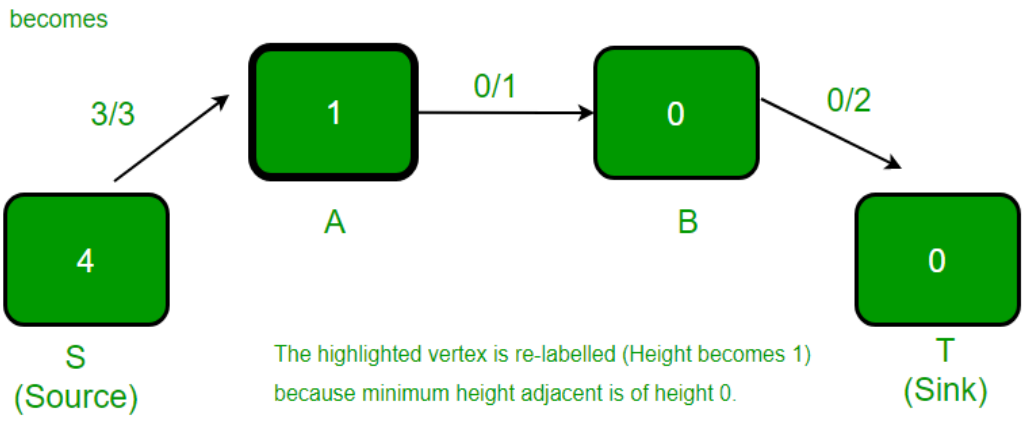
1. Initial given flow graph.



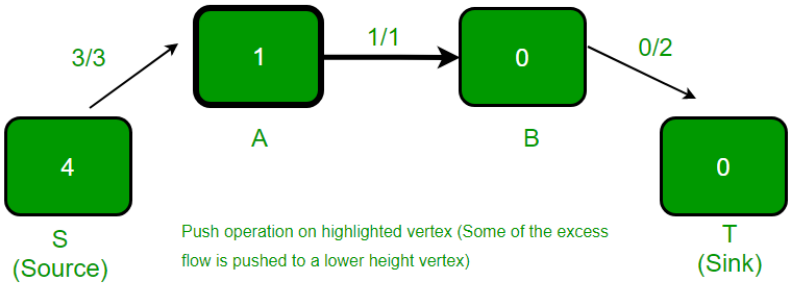
2. **After PreFlow operation**. In residual graph, there is an edge from A to S with capacity 3 and no edge from S to A.



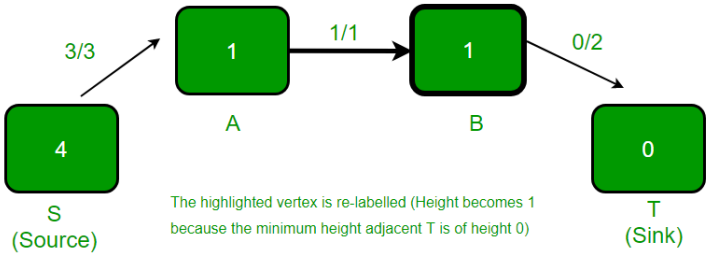
3. The highlighted vertex is relabeled (height becomes 1) as it has excess flow and there is no adjacent with smaller height. The new height is equal to minimum of heights of adjacent plus 1. In residual graph, there are two adjacent of vertex A, one is S and other is B. Height of S is 4 and height of B is 0. Minimum of these two heights is 0. We take the minimum and add 1 to it.



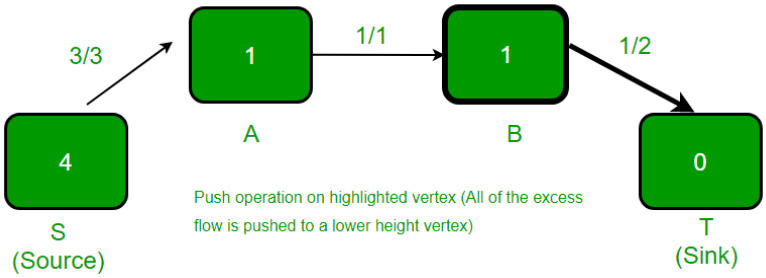
4. The highlighted vertex has excess flow and there is an adjacent with lower height, so push() happens. Excess flow of vertex A is 2 and capacity of edge (A, B) is 1. Therefore, the amount of pushed flow is 1 (minimum of two values).



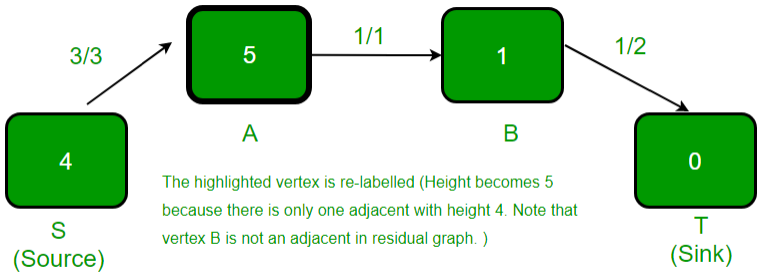
5. The highlighted vertex is relabeled (height becomes 1) as it has excess flow and there is no adjacent with smaller height.



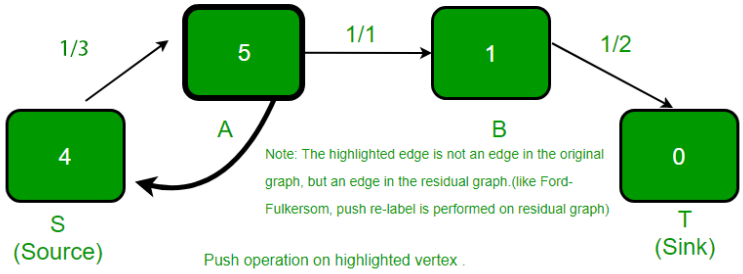
6. The highlighted vertex has excess flow and there is an adjacent with lower height, so flow() is pushed from B to T.



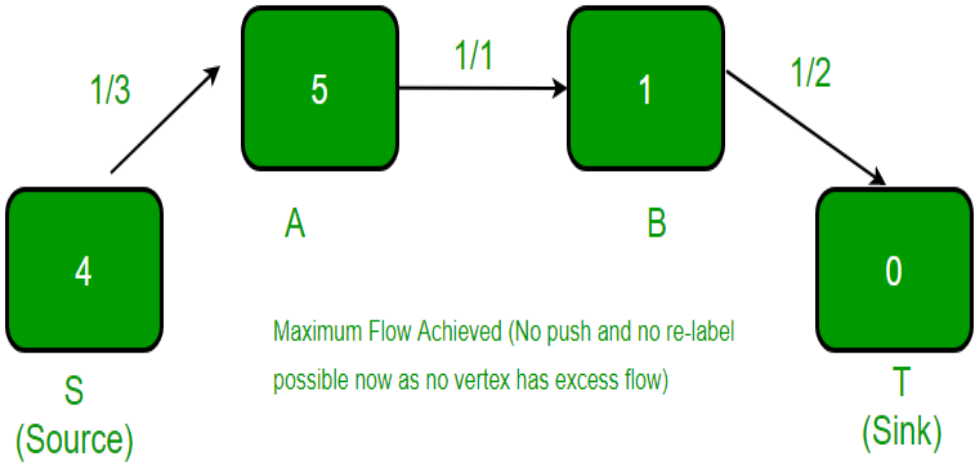
7. The highlighted vertex is relabeled (height becomes 5) as it has excess flow and there is no adjacent with smaller height.



8. The highlighted vertex has excess flow and there is an adjacent with lower height, so push() happens.



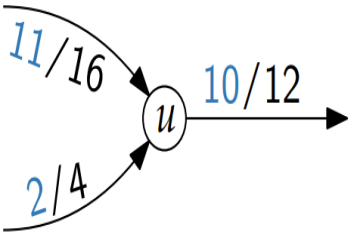
9. The highlighted vertex is relabeled (height is increased by 1) as it has excess flow and there is no adjacent with smaller height.



Preflow, excess flow and height

A **preflow** in G is a real-value function $f\colon V \times V \to \mathbb{R}$ that satisfies the capacity constraint and, for all $u \in V \setminus \{s\}$,

- $\sum_{v \in V} f(v,u) - \sum_{v \in V} f(u,v) \geq 0$.



The **excess flow** of a vertex u is

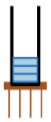
- $e(u) = \sum_{v \in V} f(v,u) - \sum_{v \in V} f(u,v)$.

$e(u) = 3$

A vertex u is called **overflowing**, when $e(u) > 0$.

For a flow network G with preflow f, a **height function** is a function $h\colon V \to \mathbb{N}$ such that

- $h(s) = |V|$,
- $h(t) = 0$, and
- $h(u) \leq h(v) + 1$ for every residual edge $(u,v) \in E_f$.



PUSH operation

PUSH(u, v)

Condition: u is overflowing, $c_f(u, v) > 0$, and $h(u) = h(v) + 1$

Effect: Push $\min(e(u), c_f(u, v))$ overflow from u to v

$\Delta \leftarrow \min(e(u), c_f(u, v))$

if $(u, v) \in E$ **then**

  $f(u.v) \leftarrow f(u, v) + \Delta$

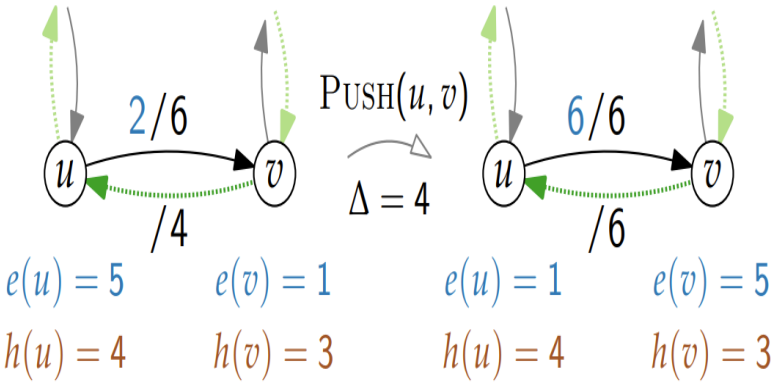
else

  $f(v.u) \leftarrow f(v, u) - \Delta$

$e(u) \leftarrow e(u) - \Delta$

$e(v) \leftarrow e(v) + \Delta$

Example.



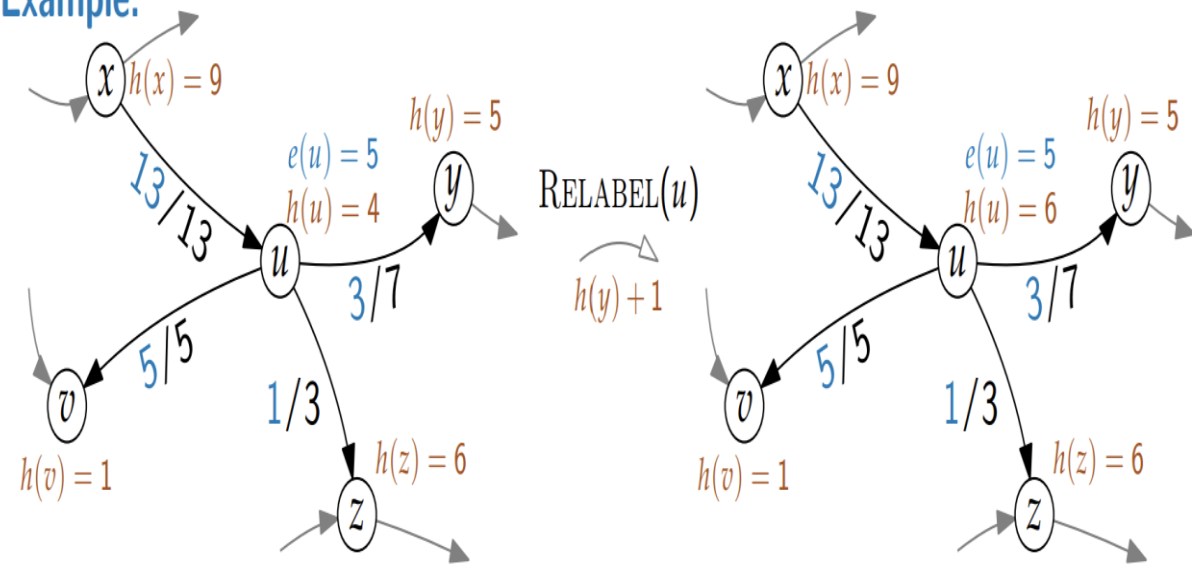
RELABEL operation

RELABEL(u)

Condition: u is overflowing and
$h(u) \leq h(v)$ for all $v \in V$ with $(u,v) \in E_f$

Effect: Increase the height of u
$h(u) \leftarrow 1 + \min\{h(v) : (u,v) \in E_f\}$

Example.



PUSH-RELABEL algorithm

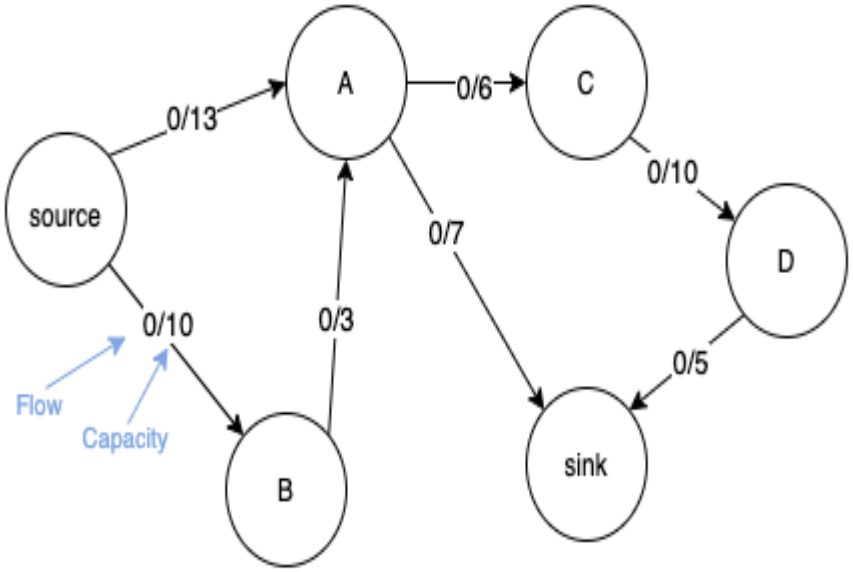
```
PUSH-RELABEL(G)
  INITPREFLOW(G, s)
  while there exists an applicable
    PUSH or RELABEL operation x do
    apply x
```

```
INITPREFLOW(G, s)
  h(v) ← 0, e(v) ← 0   ∀v ∈ V
  h(s) ← |V|
  f(u, v) ← 0   ∀(u, v) ∈ E
  for each v adjacent to s do
    f(s, v) ← c(s, v)
    e(v) ← c(s, v)
```

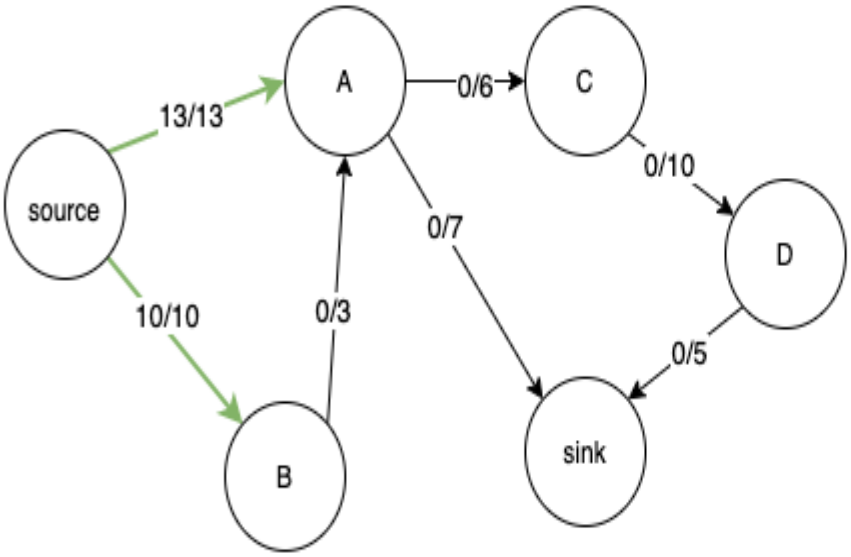
- initialises heights
- pushes max flow over all out-going edges of s

Let's understand the working of Push Relabel algorithm from a working example

1. Take the following graph

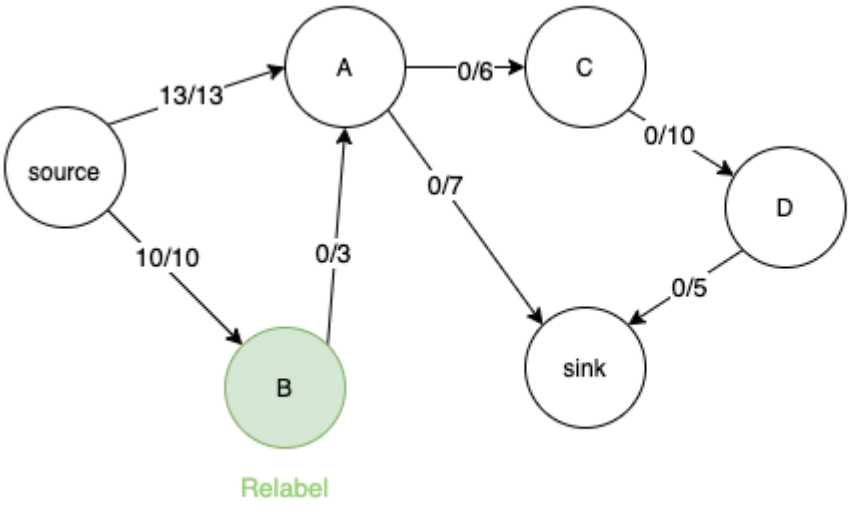


2.Initialise the graph by setting heights and excess flow



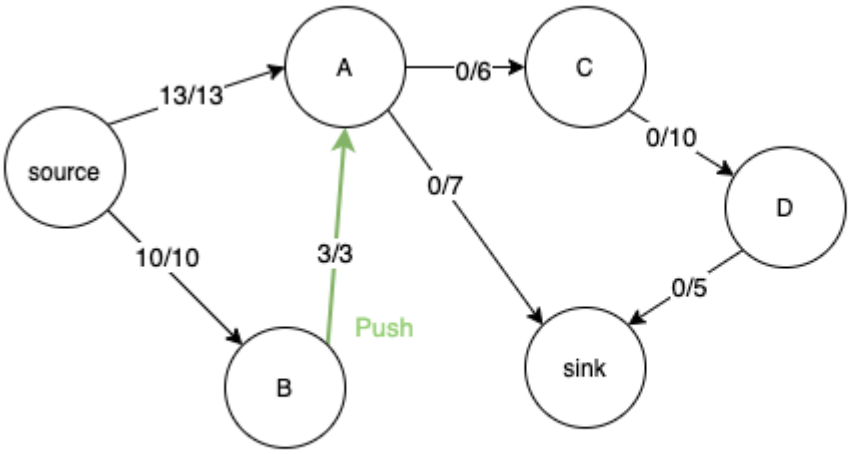
Node	Height	Excess Flow
source	6	
A	0	13
B	0	10
C	0	
D	0	
sink	0	

3.Consider vertex B. It cannot transfer its excess flow as its adjacent node A has the same height. So we relabel it.



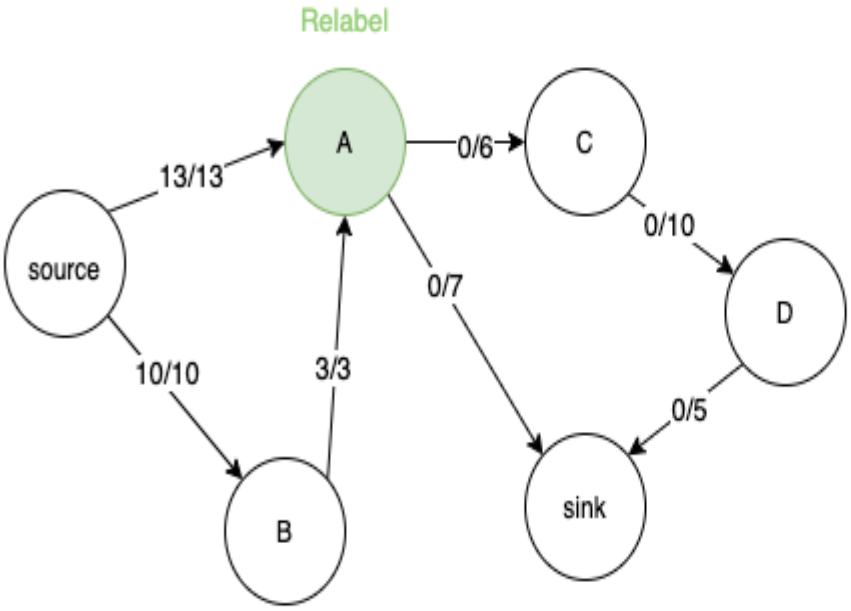
Node	Height	Excess Flow
source	6	-
A	0	13
B	1	10
C	0	
D	0	
sink	0	-

4.Now B can push its excess flow to A

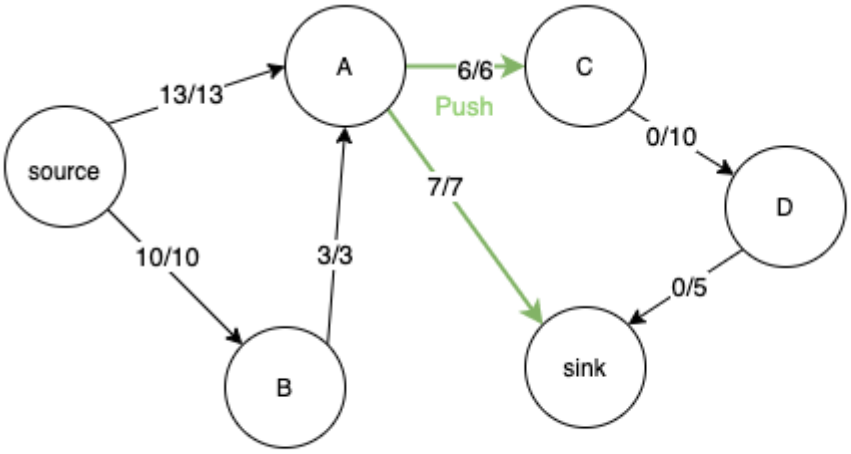


Node	Height	Excess Flow
source	6	-
A	0	16
B	1	7
C	0	
D	0	
sink	0	-

5. Similarly consider node A. We relabel its height to 1 so that it can transfer flow to its adjacent nodes C and sink.

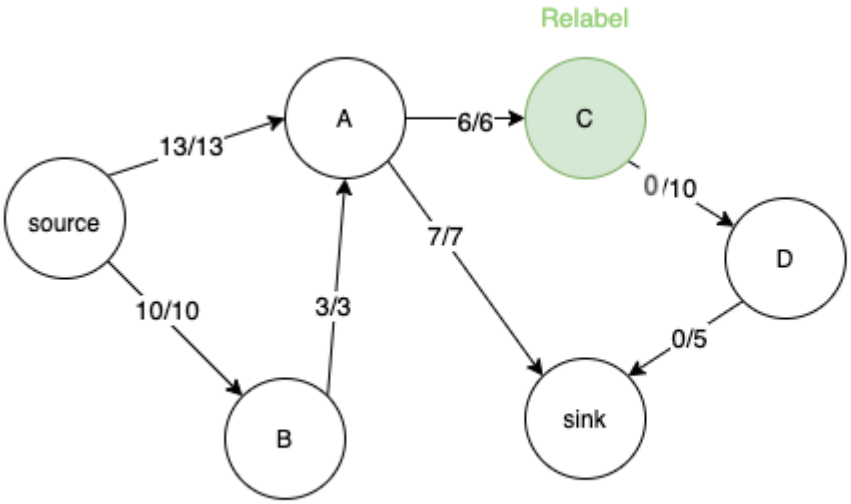


Node	Height	Excess Flow
source	6	-
A	1	16
B	1	7
C	0	
D	0	
sink	0	-



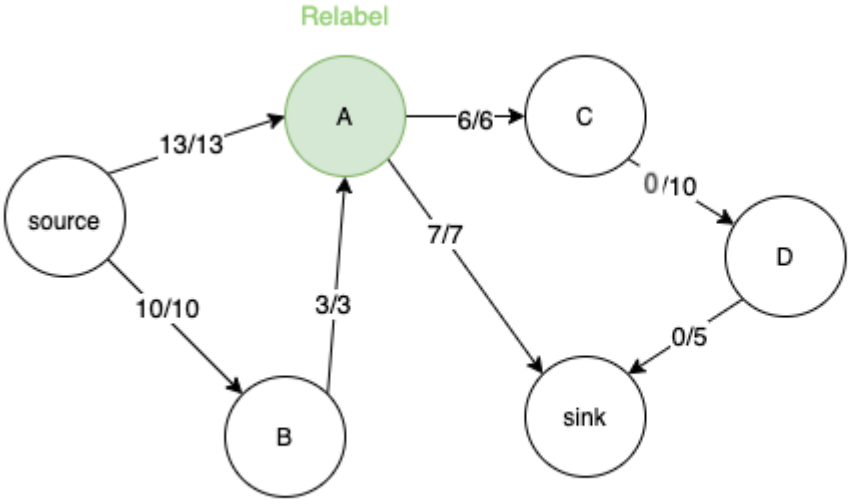
Node	Height	Excess Flow
source	6	-
A	1	3
B	1	7
C	0	6
D	0	
sink	0	-

6. Now we consider node C and relabel its height, but now flow cannot travel from A to C since they are at the same height.



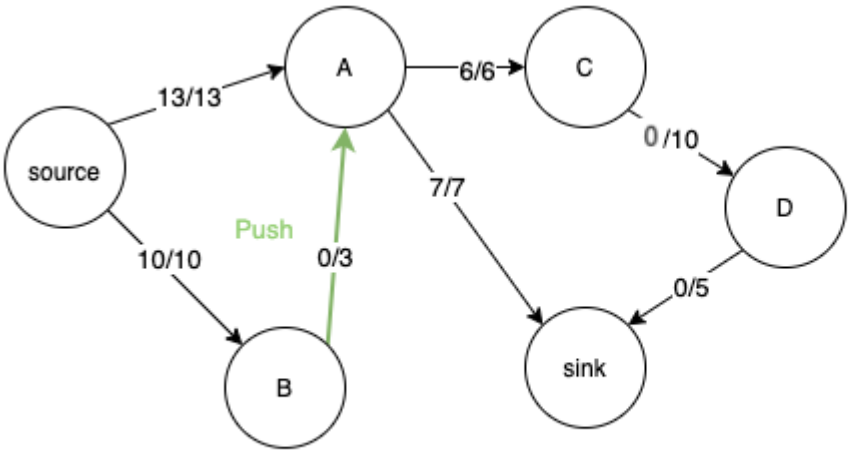
Node	Height	Excess Flow
source	6	-
A	1	3
B	1	7
C	1	0
D	0	0
sink	0	-

7.Hence we relabel node A



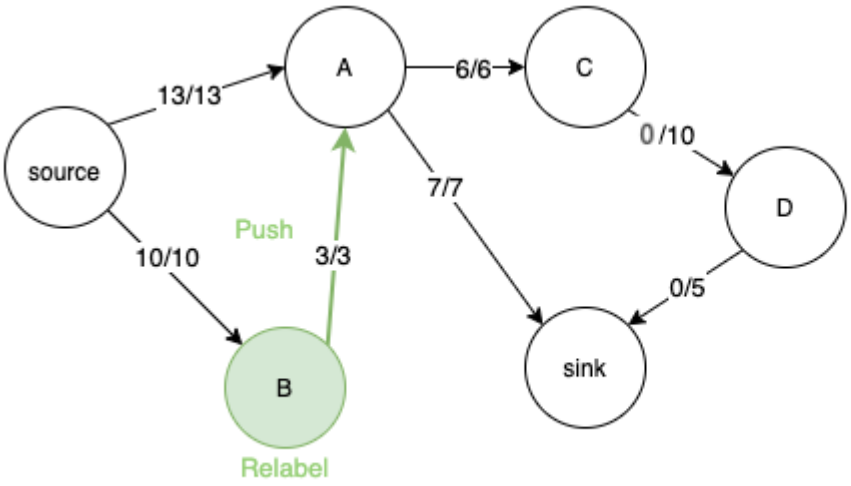
Node	Height	Excess Flow
source	6	-
A	2	3
B	1	7
C	1	0
D	0	0
sink	0	-

8. Since height of A is greater than height of B, it can now push back the extra flow to B



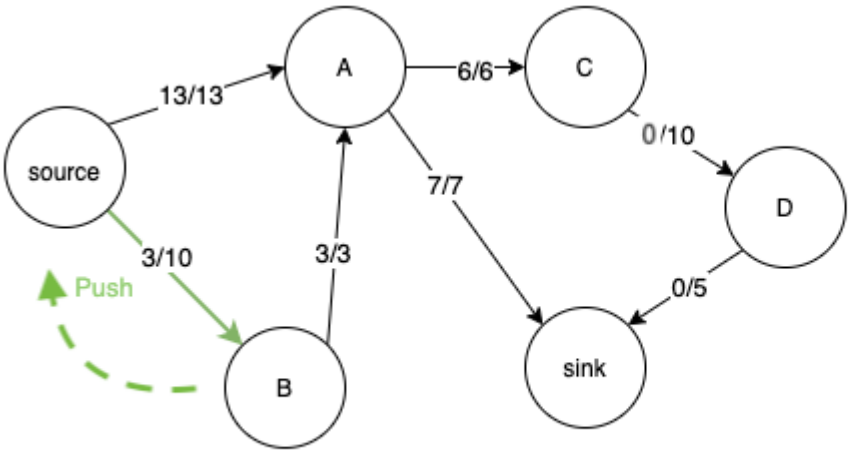
Node	Height	Excess Flow
source	6	-
A	2	0
B	1	10
C	1	0
D	0	0
sink	0	-

9. Since B has no other edge to transfer the flow, we relabel it again and transfer the extra flow back to A



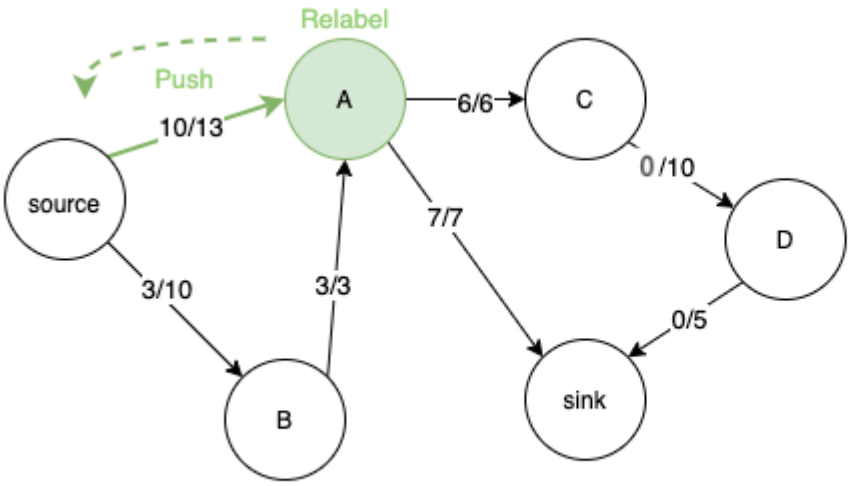
Node	Height	Excess Flow
source	6	-
A	2	3
B	3	7
C	1	0
D	0	0
sink	0	-

10. This to and fro operation between A and B happens till height of B is greater than source node. Now we can push back the extra flow back to source node



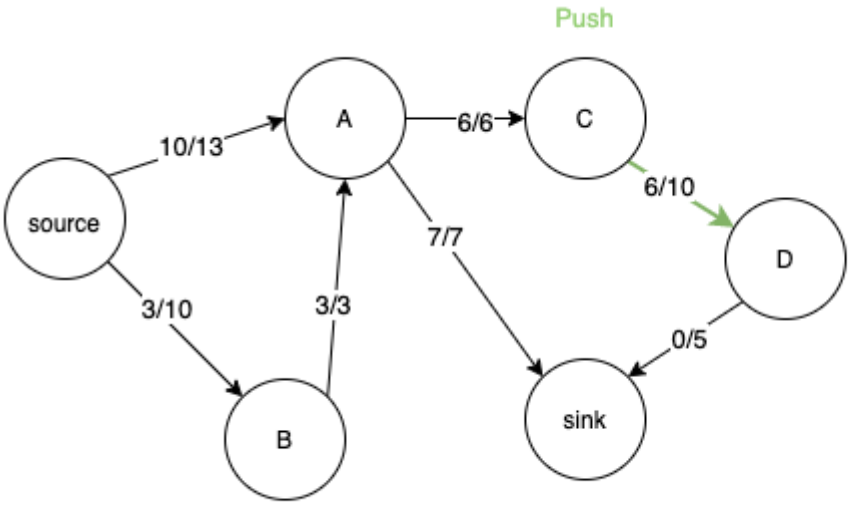
Node	Height	Excess Flow
source	6	-
A	6	3
B	7	0
C	1	0
D	0	0
sink	0	-

11. Similarly, when we relabel A, its height also becomes greater than source and we can push back extra flow to the source. Now both A and B nodes have 0 extra flow.



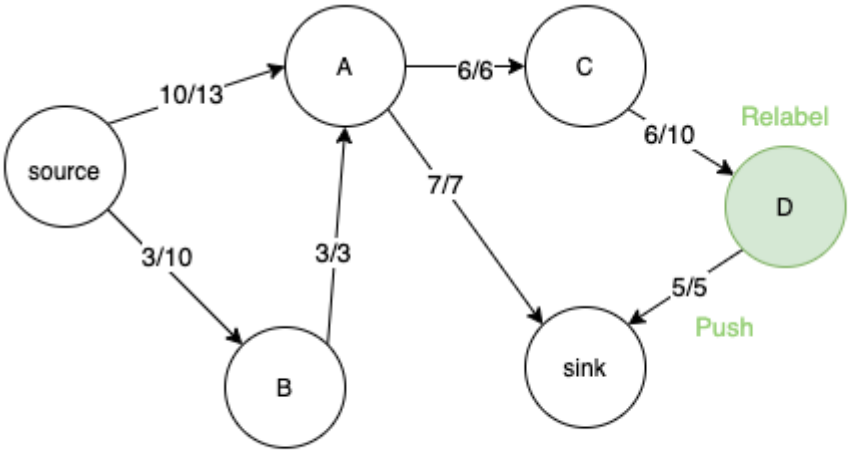
Node	Height	Excess Flow
source	6	-
A	8	0
B	7	0
C	1	0
D	0	0
sink	0	-

12.Now we consider node C and push its extra flow to node D.



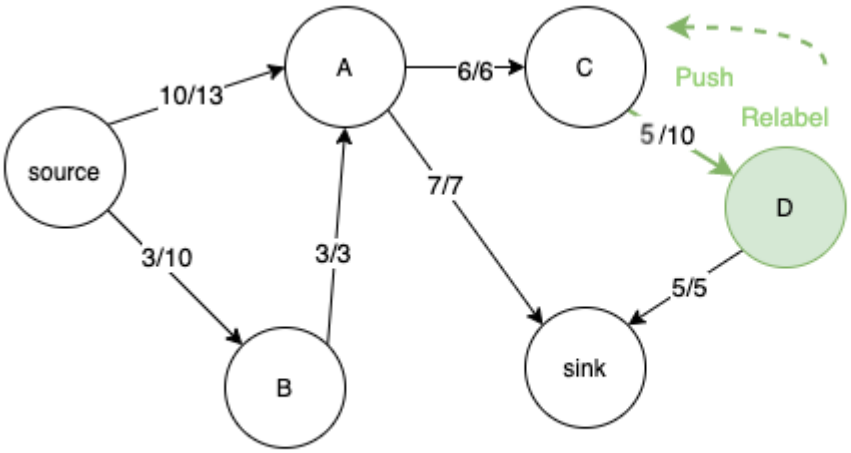
Node	Height	Excess Flow
source	6	-
A	8	0
B	7	0
C	1	0
D	0	6
sink	0	-

13. We relabel D and push its extra flow to sink. However we are still left with 1 unit of extra flow in D



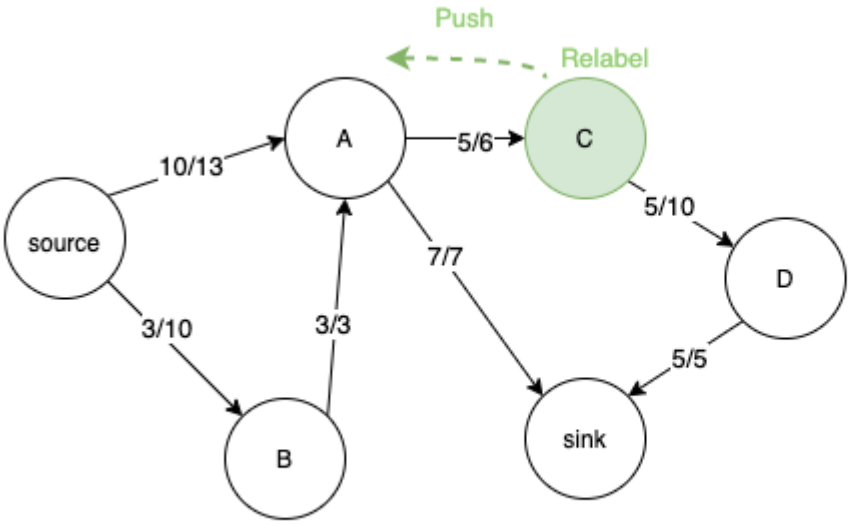
Node	Height	Excess Flow
source	6	-
A	8	0
B	7	0
C	1	0
D	1	1
sink	0	-

14. So, we relabel D again and push back the extra flow to C.



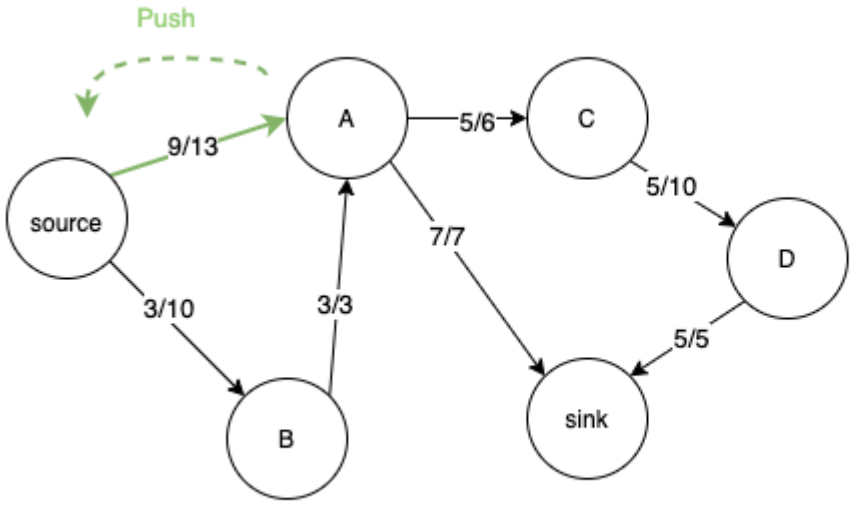
Node	Height	Excess Flow
source	6	-
A	8	0
B	7	0
C	1	1
D	2	0
sink	0	-

15.C again gets relabeled and the flow is pushed back to D. Flow moves to and fro between D and C till height of C becomes greater than A. Then C can push back 1 unit of extra flow to



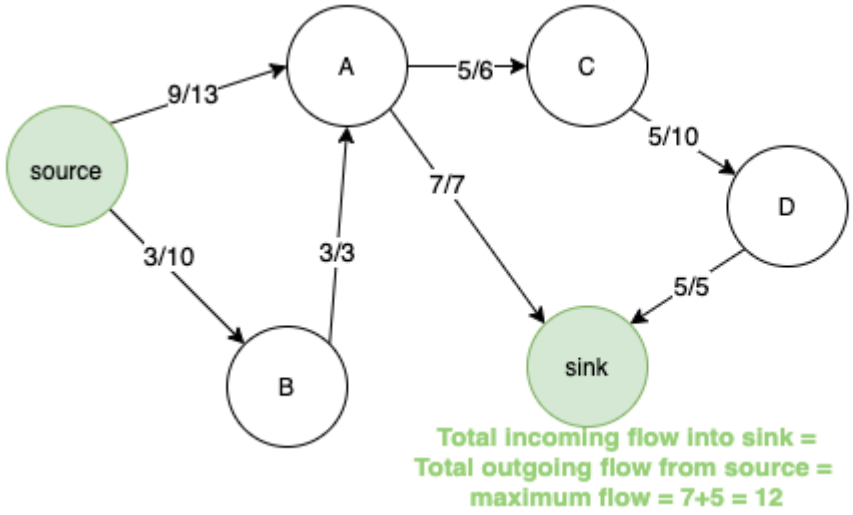
Node	Height	Excess Flow
source	6	-
A	8	1
B	7	0
C	9	0
D	8	0
sink	0	-

16. Since height of A is greater than source, A also pushes back the extra flow to source. Now the extra flow at all the nodes have become 0. The algorithm terminates here.



Node	Height	Excess Flow
source	6	-
A	8	0
B	7	0
C	9	0
D	8	0
sink	0	-

17. Maximum flow can be calculated by summing the total outgoing flow from the source or total incoming flow in the sink. In this case it is equal to 12



Node	Height	Excess Flow
source	6	-
A	8	0
B	7	0
C	9	0
D	8	0
sink	0	-

Correctness

Part 1.

If the algorithm terminates, the preflow is maximum flow.

- If an overflowing vertex exists, the algorithm can continue.
- The algorithm maintains f as a preflow and h as a height function.
- Sink t is not reachable from source s in G_f.

Part 2.

The algorithm terminates and the heights stay finite.

- Find upper bound on heights.
- Find upper bound for calls of RELABEL.
- Find upper bound for calls of PUSH.

Continuation

Lemma 1.
If a vertex u is overflowing, either a push or a relabel operation applies to u.

Proof.

Assuming $h(u)$ is valid, we have

- $h(u) \leq h(v) + 1$ for all v with $(u, v) \in E_f$.

If no push operation valid for $(u, v) \in E_f$, then

- $h(u) \leq h(v)$ for all v with $(u, v) \in E_f$.

Therefore, RELABEL(u) is applicable.

Height function:

- $h(s) = |V|$,
- $h(t) = 0$, and
- $h(u) \leq h(v) + 1$ for every residual edge $(u, v) \in E_f$.

PUSH(u, v)

Condition: u is overflowing, $c_f(u, v) > 0$, and $h(u) = h(v) + 1$
$\Delta \leftarrow \min(e(u), c_f(u, v))$
if $(u, v) \in E$ **then**
  $f(u.v) \leftarrow f(u, v) + \Delta$
else
  $f(v.u) \leftarrow f(v, u) + \Delta$
$e(u) \leftarrow e(u) - \Delta$
$e(v) \leftarrow e(v) + \Delta$

RELABEL(u)

Condition: u is overflowing, $h(u) \leq h(v)\ \forall v \in V$ with $(u, v) \in E_f$
$h(u) \leftarrow 1 + \min\{h(v) : (u, v) \in E_f\}$

Maintaining the preflow

Lemma 2.
The push-relabel algorithm maintains a preflow f.

Proof.

- InitPreflow initialises a preflow f. ✓
- Relabel(u) doesn't affect f. ✓
- Push(u, v) maintains f as a preflow. ✓

Height function:

- $h(s) = |V|$,
- $h(t) = 0$, and
- $h(u) \leq h(v) + 1$ for every residual edge $(u, v) \in E_f$.

Push(u, v)

Condition: u is overflowing,
$c_f(u, v) > 0$, and $h(u) = h(v) + 1$
$\Delta \leftarrow \min(e(u), c_f(u, v))$
if $(u, v) \in E$ **then**
  $f(u.v) \leftarrow f(u, v) + \Delta$
else
  $f(v.u) \leftarrow f(v, u) + \Delta$
$e(u) \leftarrow e(u) - \Delta$
$e(v) \leftarrow e(v) + \Delta$

Relabel(u)

Condition: u is overflowing,
$h(u) \leq h(v)\ \forall v \in V$ with $(u, v) \in E_f$
$h(u) \leftarrow 1 + \min\{h(v) : (u, v) \in E_f\}$

Maintaining the height function

Lemma 3.
The push-relabel algorithm maintains h as a height function.

Proof.

- InitPreflow initialises h as a height function. ✓
- Push(u, v) leaves h a height function. ✓
  - If (v, u) added to E_f, then $h(v) = h(u) - 1 < h(u) + 1$.
  - If (u, v) removed from E_f, then ✓.
- Relabel(u) leaves h a height function. ✓
  - $(u, v) \in E_f$, then $h(u) \leq h(v) + 1$
  - $(w, u) \in E_f$, then $h(w) < h(u) + 1$

Height function:

- $h(s) = |V|$,
- $h(t) = 0$, and
- $h(u) \leq h(v) + 1$ for every residual edge $(u, v) \in E_f$.

Push(u, v)

Condition: u is overflowing,
$c_f(u, v) > 0$, and $h(u) = h(v) + 1$
$\Delta \leftarrow \min(e(u), c_f(u, v))$
if $(u, v) \in E$ **then**
  $f(u.v) \leftarrow f(u, v) + \Delta$
else
  $f(v.u) \leftarrow f(v, u) + \Delta$
$e(u) \leftarrow e(u) - \Delta$
$e(v) \leftarrow e(v) + \Delta$

Relabel(u)

Condition: u is overflowing,
$h(u) \leq h(v)\ \forall v \in V$ with $(u, v) \in E_f$
$h(u) \leftarrow 1 + \min\{h(v) : (u, v) \in E_f\}$

Reachability of the sink

Lemma 4.
During the push-relabel algorithm, there is no path from s to t in G_f.

Height function:
- $h(s) = |V|$,
- $h(t) = 0$, and
- $h(u) \leq h(v) + 1$ for every residual edge $(u, v) \in E_f$.

Proof.
Suppose there is a path $s = v_0, v_1, \ldots, v_k = t$ in G_f.
Then

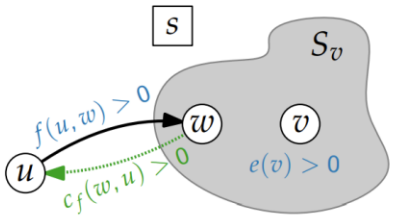
- $(v_i, v_{i+1}) \in E_f$ for $0 \leq i \leq k-1$, and
- $h(v_i) \leq h(v_{i+1}) + 1$ for $0 \leq i \leq k-1$.

$$\Rightarrow h(s) \leq h(t) + k = k$$

But then and since $k \leq |V| - 1$, follows $h(s) < |V|$. ✗

Reachability of source in residual graph

Lemma 6.
There is a path from every overflowing vertex v to s in G_f.



Proof.

- $S_v \leftarrow$ vertices reachable from v in G_f.
- Suppose $v \notin S_v$.
- Since f a preflow and $s \notin S_v$, we have $\sum_{w \in S_v} e(w) \geq 0$.
- Since $v \in S_v$, we even have $\sum_{w \in S_v} e(w) > 0$.
- There is edge (u, w) with $u \notin S_v, w \in S_v$ and $f(u, w) > 0$.
- But then $c_f(w, u) > 0$, meaning u is reachable from v. ✗

Upper bound on height and RELABEL operations

Lemma 7.
During the push-relable algorithm, we have $h(v) \leq 2|V| - 1$ for all $v \in V$.

Proof.

- Statement holds after initialisation.
- Let v be an overflowing vertex that is relabeled.
- By Lemma 6, there is a path $v = v_0, v_1, \ldots, v_k = s$ in G_f.
- Then $h(v_i) \leq h(v_{i+1}) + 1$ for $0 \leq i \leq k - 1$.
- Since $k \leq |V| - 1$, we have $h(v) \leq h(s) + k \leq 2|V| - 1$.

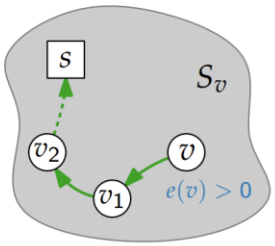
Corollary 8.
The push-relable algorithm executes at most $2|V|^2$ RELABEL operations.

Height function:

- $h(s) = |V|$,
- $h(t) = 0$, and
- $h(u) \leq h(v) + 1$ for every residual edge $(u, v) \in E_f$.

RELABEL(u)

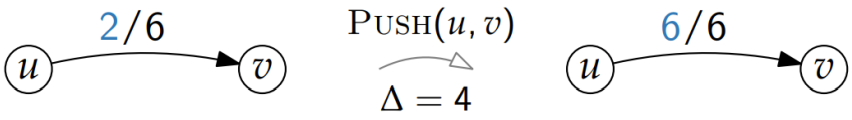
Condition: u is overflowing,
$h(u) \leq h(v)\ \forall v \in V$ with $(u, v) \in E_f$
$h(u) \leftarrow 1 + \min\{h(v) : (u, v) \in E_f\}$



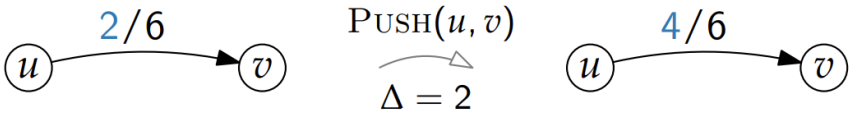
Saturating and unsaturating PUSH operations

The operation PUSH(u, v) is

- **saturating**, if afterwards $c_f(u, v) = 0$,



- and **unsaturating** otherwise.



Upper bound on saturating PUSH operations

Lemma 9.
The push-relable algorithm executes at most $2|V||E|$ saturating PUSH operations.

Proof.

- Consider saturating $\text{PUSH}(u, v)$
  - $h(u) = h(v) + 1$
- For another saturating $\text{PUSH}(u, v)$, first $\text{PUSH}(v, u)$ necessary
  - $h(v) = h(u) + 1$ necessary
- After another saturating $\text{PUSH}(u, v)$, both $h(u)$ and $h(v)$ have increased by at least two.
- But by Lemma 6, $h(u) \leq 2|V| - 1$ and $h(v) \leq 2|V| - 1$.
- There are at most $2|V| - 1$ sat. PUSH operations for edge (u, v).

$\text{PUSH}(u, v)$
Condition: u is overflowing,
$c_f(u, v) > 0$, and $h(u) = h(v) + 1$
$\Delta \leftarrow \min(e(u), c_f(u, v))$
if $(u, v) \in E$ **then**
  $f(u.v) \leftarrow f(u, v) + \Delta$
else
  $f(v.u) \leftarrow f(v, u) + \Delta$
$e(u) \leftarrow e(u) - \Delta$
$e(v) \leftarrow e(v) + \Delta$

$\text{PUSH}(u, v)$
...
$\text{PUSH}(v, u)$
...
$\text{PUSH}(u, v)$
...

Upper bound on unsaturating PUSH operations

Lemma 10.
The push-relable algorithm executes at most $4|V|^2|E|$ unsaturating PUSH ops.

Proof.

- Consider $\mathcal{H} = \sum_{\substack{v \in V \setminus \{s,t\}, \\ v \text{ overflowing}}} h(v)$.
- After initialisation and at the end $\mathcal{H} = 0$.
- A saturating PUSH increases $\mathcal{H}$ by at most $2|V| - 1$.
- By Lemma 8, all saturating PUSH ops. increases $\mathcal{H}$ by at most $(2|V| - 1) \cdot 2|V||E|$.
- By Lemma 7, all RELABEL ops increases $\mathcal{H}$ by at most $(2|V| - 1) \cdot |V|$.
- An unsaturating $\text{PUSH}(u, v)$ decrease $\mathcal{H}$ by at least 1, since $h(u) - h(v) \geq 1$.

$\text{PUSH}(u, v)$
Condition: u is overflowing,
$c_f(u, v) > 0$, and $h(u) = h(v) + 1$
$\Delta \leftarrow \min(e(u), c_f(u, v))$
if $(u, v) \in E$ **then**
  $f(u.v) \leftarrow f(u, v) + \Delta$
else
  $f(v.u) \leftarrow f(v, u) + \Delta$
$e(u) \leftarrow e(u) - \Delta$
$e(v) \leftarrow e(v) + \Delta$

Termination of the algorithm

Theorem 5.
If the push-relabel algorithm terminates, the computed preflow f is a maximum flow.

Theorem 11.
The push-relabel algorithm terminates after $\mathcal{O}(|V|^2|E|)$ valid PUSH or RELABEL ops.